```
#fibonacci

.data
ent: .asciiz "Please enter how many fibonacci numbers you want:"
message: .asciiz " number of fibonacci series is stored at memory location "
series: .asciiz "fibonacci series: "

.text
main:
li $v0, 4
la $a0, ent
syscall

li $v0, 5
syscall

move $s0, $v0 #here we have stored n.
ble $s0, $0, end #if user input is non-positive print nothing

#allocating space to save
#li $t0, 4
#mul $a0, $t0, $s0
sll $a0, $s0, 2
addi $a0, $a0, 8
li $v0, 9
syscall
move $s7, $v0 #base address of the number store.
move $s6, $s7 #the memory pointer we will advance



#initialising with first two numbers
li $s1, 0
sw $s1, 0($s6)
li $s2, 1
#addi $s6, $s6, 4
sw $s2, 4($s6)
li $s3, 0 #counter


li $v0, 1
addi $a0, $s3, 1
syscall
li $v0, 4
la $a0, message
syscall
li $v0, 1
move $a0, $s6
syscall
li $v0, 11
li $a0, 10 #newline char
syscall

addi $s3, $s3, 1
addi $s6, $s6, 8 #this command here so 0 fibonacci printed correctly.

loop:
beq $s3, $s0, boop

#print the new number
li $v0, 1
addi $t1, $s3, 1 #counter starts from 0.
move $a0, $t1
syscall
li $v0, 4
la $a0, message
syscall
```

```
li $v0, 1
addi $t3, $s6, -4 #to display memory correctly adding this step
move $a0, $t3
syscall
li $v0, 11
li $a0, 10 #newline char
syscall

move $t0, $s2
add $s2, $s2, $s1
move $s1, $t0
sw $s2, 0($s6)
addi $s6, $s6, 4 #advancing the memory pointer
addi $s3, $s3, 1
j loop

#now printing the final series of numbers
boop:
li $v0, 4
la $a0, series
syscall
li $s3, 0
goop:
beq $s3, $s0, end
lw $a0, 0($s7)
li $v0, 1
syscall
#special beq check to prevent the last comma
addi $t1, $s3, 1
beq $t1, $s0, end
li $v0, 11
li $a0, ','
syscall
#updating loop variables
addi $s3, $s3, 1
addi $s7, $s7, 4
j goop



end:
li $v0, 10
syscall
```